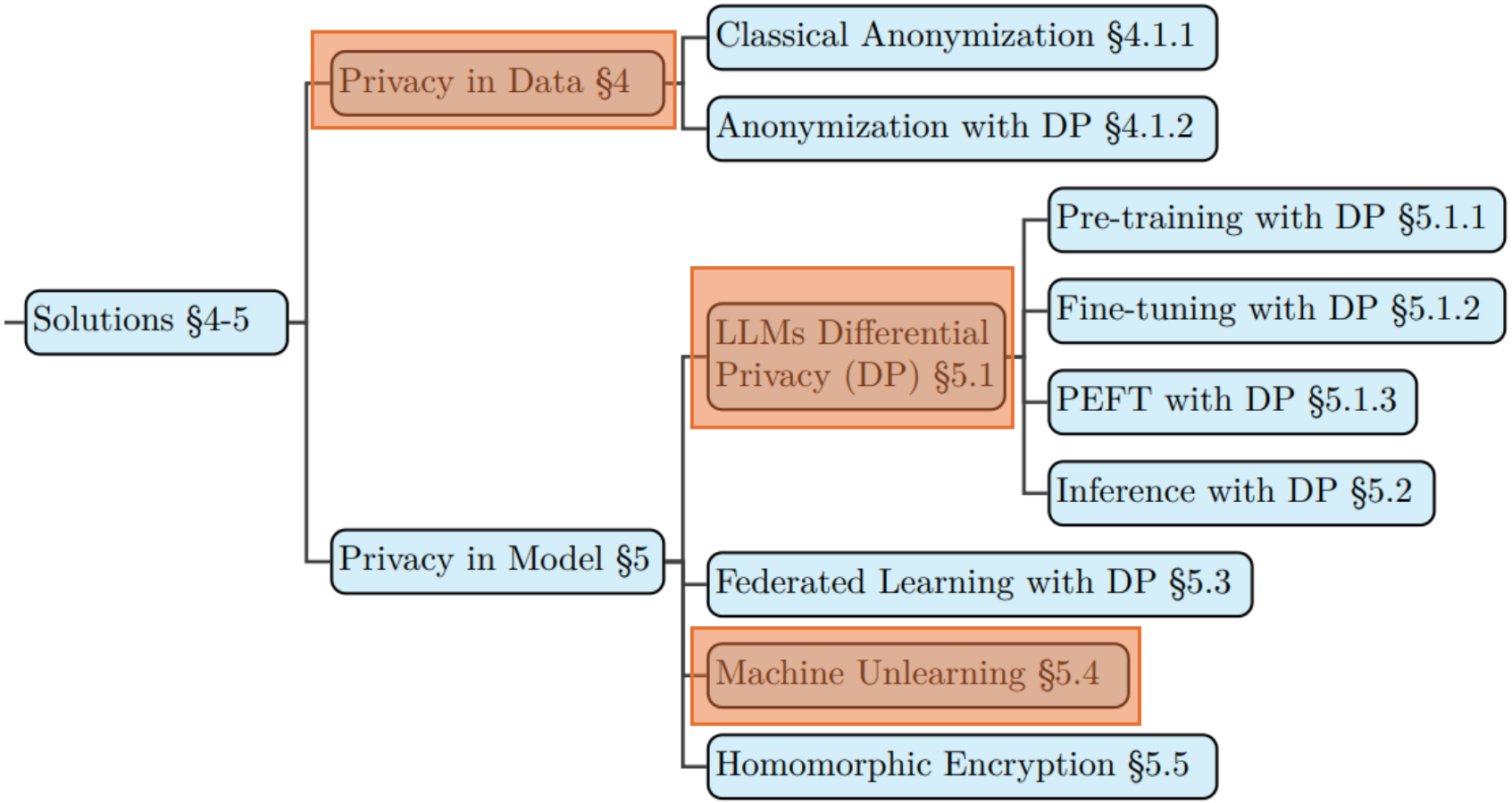
Defenses Against Privacy Attacks

Lecture Goals



Broad Goals

- Differential Privacy
  - Provable solution to defend against privacy attacks
  - May sacrifice utility of a model
  - Privacy-by-design!
- Machine Unlearning
  - Heuristic solution
  - Different algorithms with different trade-offs
  - Post-hoc solution

Privacy in Data

Briefly

Discussion on Data Anonymization

- Wait wait, why do we need a special privacy-preserving algorithm at all? Surely the easiest solution is to just *remove* any private information by anonymizing the data?
- In practice:
  - For large scale dataset, it is error prone and often simply not done
  - And even when it is done, is not that easy! Netflix anonymized the training set for its recommender challenge by replacing all names with random ids.
  - Turns out that by **linking** this dataset with public IMDB review ratings, researchers were able to deanonymise people.
  - How?

Discussion on Data Anonymization

- Different attributes could be anonymized. While effectively masking **PII** – like names, emails, and social security numbers – is necessary, some features that are not exactly identifiers can be used in combination to identify an individual: those features are called **quasi-identifiers**.
- Quasi-identifiers **are apparently non-personal attributes** – like gender, date of birth, and zip codes – that can be used in combination **with external resources** to identify a person from its anonymized records.

Classical Anonymization: K-anonymity

- Anonymization aims to make the re-identification of an individual impossible, that is, the process of reconstructing the identity of an individual given a record that contains their information.
- The definition of k-anonymity (Sweeney, 1997) ensures that a quasi-identifier cannot be used for re-identification. To respect k-anonymity means ensuring that **data cannot be re-identified to fewer than k individuals**.
- This foundational concept has spurred extensive research, leading to a plethora of anonymization techniques.

Classical Anonymization: K-anonymity

- A dataset is said to have k-anonymity if any given record in the dataset is **indistinguishable from at least k - 1 other records** with respect to certain identifying attributes called **quasi-identifiers**.
- Several implementation techniques were proposed:
  - generalization (modifying attribute values to less specific but accurate representations)
  - suppression (i.e., entirely removing data that are outliers or too identifiable),
  - minimal Generalization (ensuring data are not generalized more than necessary to meet k-anonymity)
- Those methods are less effective with unstructured information.

Classical Anonymization: General pipeline for text

- After any necessary preprocessing phase, the first challenge to solve is to identify the entities that may cause a violation of privacy. This can be done via a Named Entity Recognition (NER) tool.
- To maintain the coherence of the text, a Co-reference Resolution (CRR) system can be used to resolve references between entities should be identified if the anonymization framework aims to ensure consistent anonymization across the document.
- Finally, private information should be replaced with anonymous information. The anonymization methods can vary. See next slide

Classical Anonymization: General pipeline for text

- Anonymization methods:
  - Suppression is used to completely remove the information
  - Tagging allows the replacement of sensitive information with artificial labels –like the one obtained by the NER system– that still retain information about the class of the data and an identifier.
  - The sensitive information can also be replaced with other textual information:
    - random substitution: uses another random entity of the same class
    - Generalization: generalization replaces an entity with a more general term
- While this pipeline is intended for general data anonymization, regardless of which model is going to be trained on it, it is specific to textual data, as it relies on some specific features of text.

Limitations of k-anonymity

- Curse of Dimensionality (Data Sparsity):
  - As the number of quasi-identifier dimensions (attributes) increases, the data becomes sparse. This means data points are spread out, making it difficult to find similar records to group together, requiring massive, often useless, generalizations.
- High Information Loss:
  - To achieve k-anonymity, a high number of dimensions necessitates extensive suppression or generalization (e.g., changing specific ages to broad ranges). This distortion often makes the data useless for analysis or machine learning.
- One wants the least disruptive generalization/suppression:
  - Guess what? It is a NP-Hard problem

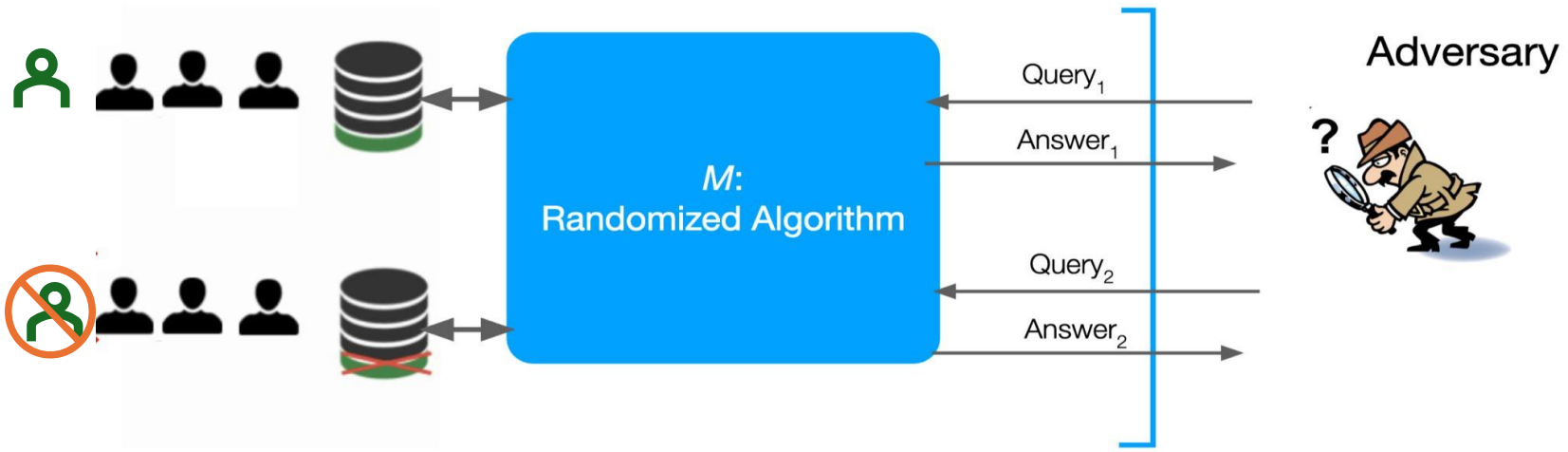
Remarks

- Every time we perform any form of statistical analysis, we leak a little information about the data.
- If I tell you that the mean of the ages of the participants of a dataset is 34, then I've revealed a bit of information about the participants.
- If we want *total* privacy, we learn nothing about the data.
- Revealing the mean age tells us less than revealing all of the participants' ages. We may want to **quantify this loss of privacy**.

https://mukulrathi.com/privacy-preserving-machine-learning/deep-learning-differential-privacy/

Differential Privacy

Differential Privacy: Intuition



- d1 and d2: neighboring datasets (identical except for 's record)
- Goal: M's output (Answer1 or Answer2) doesn't enable adversary distinguish between d1 and d2
- Answer1 or Answer2 is essentially the same whether or not 's record is in d1

Differential Privacy: Intuition

"Differential privacy describes a promise, made by a data holder, or curator, to a data subject: "you will not be affected, adversely or otherwise, by allowing your data to be used in any study or analysis, no matter what other studies, data sets, or information sources, are available.“

DP fundamental goal

- enable the extraction of useful insights from data
- minimizing the potential impact on any individual’s privacy within that dataset.

When a system or algorithm is differentially private, the inclusion or exclusion of any individual’s data should not significantly affect the overall outcome or conclusions of the analysis.

Dwork et al,. https://doi.org/10.1561/0400000042

Example

- Training a model on a cancer dataset D, and we ask it to predict if a particular patient Bob has cancer. We'll call this entire mechanism (training a model and then predicting whether Bob has cancer) M.
- $P\big(M(D) = \text{“Bob has cancer”}\big) = 0.55$
- If we train a model on the same dataset *augmented with Bob's records* and it improves from 55% to 57% confident,
- $P\big(M(D + Bob) = \text{“Bob has cancer”}\big) = 0.57$
- then maybe Bob's data says he has cancer, but maybe it's just that the model got more confident overall with more data. Bob's privacy comes from this *plausible deniability* of him having cancer.
- What if it actually became 80% confident Bob has cancer?
- $P\big(M(D + Bob) = \text{“Bob has cancer”}\big) = 0.80$
- Just by adding Bob to the dataset, the outcome "Bob has cancer" has become *significantly more likely*, so it's pretty clear that Bob does in fact have cancer, and the mechanism M **leaked information** about Bob!
- We can quantify the **privacy loss** as the log of the ratio of the probabilities before and after Bob was added to the dataset: i.e.
- $\log \frac{P\big(M(D+Bob)\big)=\text{“Bob has cancer”}\big)}{P\big(M(D)=\text{“Bob has cancer”}\big)}$.

https://mukulrathi.com/privacy-preserving-machine-learning/deep-learning-differential-privacy/

Example

- Plug:
- $\log \frac{0.57}{0.55} = 0.0357$
- $\log \frac{0.80}{0.55} = 0.375$
- We can *quantifiably* say that the second outcome has **10x the privacy loss**.
- The key idea behind **differential privacy** is to *limit this difference in performance*: the model should make a make a similar inference about me whether or not I individually opt-in to the dataset, thus preserving my privacy.
- In the equations that follow that formally define differential privacy, we look at datasets D and D' that differ in one data record (i.e. we add/remove a record)

https://mukulrathi.com/privacy-preserving-machine-learning/deep-learning-differential-privacy/

Differential Privacy: Definition

- A randomized algorithm M with domain $N^{|X|}$ is (ε, δ)-differentially private if for all $S \subseteq \text{Range}(M)$ and for all $x, y \in N^{|X|}$ such that
- $\|x - y\|_1 \leq 1$:

$$\Pr[M(x) \in S] \leq \exp(\varepsilon) \Pr[M(y) \in S] + \delta$$

- For any two datasets x and y that differ by **only one element** (these are often referred to as "adjacent datasets"),
- and for any subset of outputs S of the algorithm's range Y,
- **the probability that the algorithm outputs something in S given x is not substantially higher than given y.**

Dwork *et al*., 2014: "*The Algorithmic Foundations of Differential Privacy*": https://www.cis.upenn.edu/~aaroth/Papers/privacybook.pdf

Differential Privacy: Definition

- A randomized algorithm M with domain $N^{|X|}$ is (ε, δ)-differentially private if for all $S \subseteq \text{Range}(M)$ and for all $x, y \in N^{|X|}$ such that
- $\|x - y\|_1 \leq 1$:

$$\Pr[M(x) \in S] \leq \exp(\varepsilon) \Pr[M(y) \in S] + \delta$$

- This means that:

  for all subsets S of the output range Y,

  the probability that the algorithm on input x gives an output in S is

  at most

  e raised to the power ε times the probability that the algorithm on input y' gives an output in S

  plus the small probability δ.

Differential Privacy: Definition

- ε Privacy loss: how hard it is to distinguish x and y? Small → better privacy
  - The use of ε in the inequality indicates that **the probability of a certain output does not increase** by **more than a multiplicative factor** of $\mathbf{e^{\varepsilon}}$ when any single individual's data is added or removed.
- The **privacy budget** is the **total loss of privacy sustained by the system during the execution**, and it can be computed by compounding the privacy loss indicated by ε over the total number of executions/iterations.
  - This is done with the composition theorems, later

Dwork *et al*., 2014: "*The Algorithmic Foundations of Differential Privacy*":https://www.cis.upenn.edu/~aaroth/Papers/privacybook.pdf

Differential Privacy: Definition

- δ is the parameter that allows for a small probability of failure of the privacy guarantee.
- Ideally, δ should be very small, close to 0, which indicates a lower chance of the algorithm failing to maintain privacy.
- When δ is 0, it is defined ε-differential privacy, which is the strongest kind of differential privacy

Dwork *et al*., 2014: "*The Algorithmic Foundations of Differential Privacy*":https://www.cis.upenn.edu/~aaroth/Papers/privacybook.pdf

Differential Privacy: Illustration

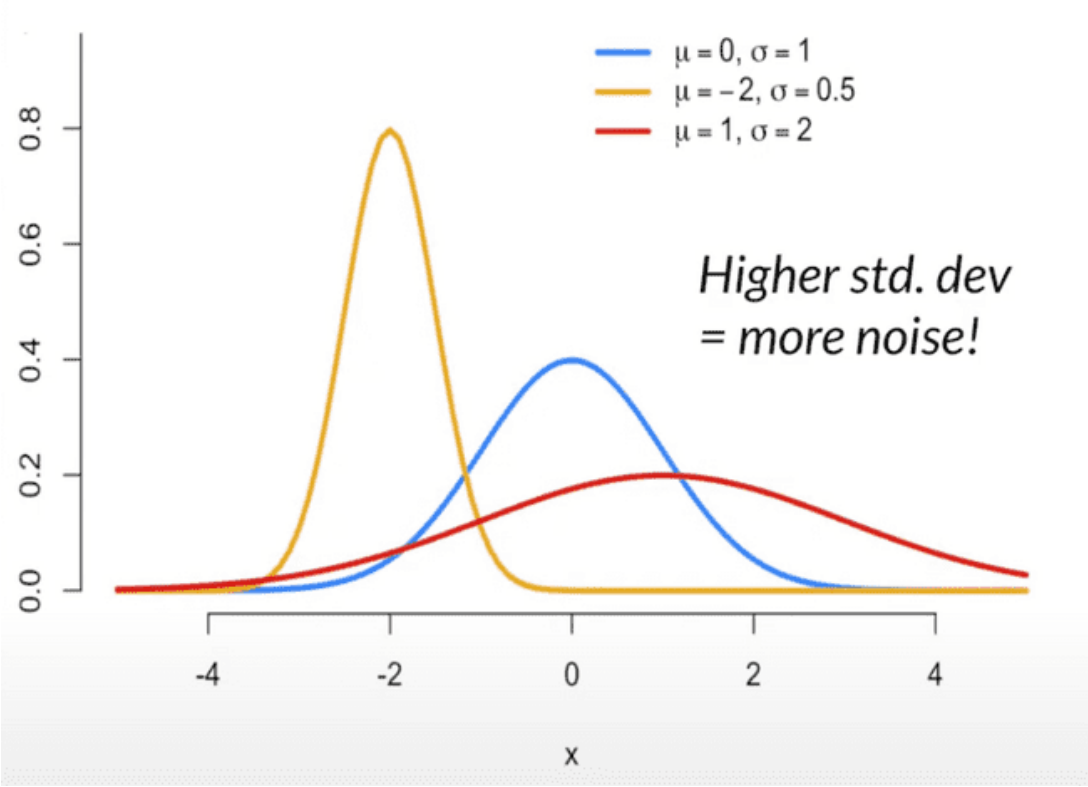
- *M*(d1) = *f*(d1) + noise
- Example for *f*(d1):
- **SELECT COUNT(*) FROM Table1 WHERE Age >=50;**
- M(d1) = 7092 + noise
- How do we add noise?
- The challenge lies in **balancing the noise added to meet differential privacy** standards without rendering the answer too noisy to be useful.

- Add random noise to query result
- Key challenge: add enough noise to improve privacy, but not too much to hurt utility
- Noise mechanisms:
  - Laplace noise
  - $$M(d) = f(d) + \frac{s * \sqrt{2 * \log \frac{1.25}{\delta}}}{\epsilon}$$

$$M(d) = f(d) + \mathrm{Lap}(\frac{s}{\epsilon})$$

Differential Privacy: Illustration

- We can limit the privacy loss by adding noise. Think of an image: the more noise you add, the fuzzier the image, so the less you learn from it.
- Every time you add a bit of random noise, you spread the range of possible values the input could have been.
- So if I take my age as 35, and then add noise sampled from a Gaussian distribution with a standard deviation of 5 - then within a 95% confidence interval my age could be anywhere between 25 and 45.
- The noise essentially smooths the probabilities and prevents one outcome from becoming much more likely.
- **The smaller your privacy budget, the more noise you need to add.**



https://mukulrathi.com/privacy-preserving-machine-learning/deep-learning-differential-privacy/

Differential Privacy With Gaussian Mechanism

Gaussian Mechanism

The Gaussian mechanism does not satisfy pure ϵ-differential privacy but does satisfy (ϵ, δ)-differential privacy. According to the Gaussian mechanism, for a function $f(x)$ which returns a number, the following definition of $F(x)$ satisfies (ϵ, δ)-differential privacy:

$$F(x) = f(x) + \mathcal{N}(0, \sigma^2)$$
$$\text{where } \sigma = \frac{\Delta f \cdot \sqrt{2\log(1.25/\delta)}}{\epsilon}$$

where Δf is the sensitivity of f, and $\mathcal{N}(0, \sigma^2)$ denotes sampling from the Gaussian (normal) distribution with mean 0 and variance σ^2. Note that here (and elsewhere in these notes), log denotes the natural logarithm.

What is the sensitivity Δf?

- Sensitivity refers to the **maximum change** in the output of a function or query **when a single individual's data** in the input dataset **is added or removed**.
  - It quantifies how much a query's result can vary with the smallest possible change in the dataset
  - This is crucial in determining the amount of noise that needs to be added to ensure differential privacy.
- A function with high sensitivity requires more noise to protect individual privacy, while a function with low sensitivity requires less.
- Sensitivity is a key factor in calibrating the noise to ensure that individual data points do not significantly affect the output, thereby maintaining privacy

What is the sensitivity Δf?

Global Sensitivity

For a function $f : \mathcal{D} \rightarrow \mathbb{R}$ mapping datasets ($\mathcal{D}$) to real numbers, the global sensitivity of f is defined as follows:

$$GS(f) = \max_{x,x':d(x,x')<=1} |f(x) - f(x')|$$

- Here, d(x, x') denotes the distance between two datasets x and x ', with the notion that two datasets are neighbours if their distance is 1 or less.
  - The way this distance is defined significantly impacts the privacy definition we achieve
- Global sensitivity is defined such that for any two neighbouring datasets x and x ' , the difference between f(x) and f(x ') does not exceed GS(f)

What is the total privacy loss?

- If we ask **enough** questions, we can finally get the true value
  - Think of the age example above, sooner or later (it depends on the std) we'll get the true answer if we query the model… No escape from Central Limit Theorem
- So the more queries we have, the larger the privacy loss
- We need to quantify this loss!
- Composition theorems allow us to estimate the total loss under multiple queries

Differential Privacy Properties

- Composition theorems that allow the computation of the exact privacy loss through many applications of the DP algorithm, which corresponds to the privacy budget.
  - **Sequential composition theorem**(s) applies when multiple differentially private mechanisms are applied to the same dataset in sequence. The theorem quantifies how the privacy guarantees accumulate over the sequence of operations
  - **Parallel composition theorem**(s) applies when multiple differentially private mechanisms are applied to disjoint subsets of the dataset. This theorem shows that the worst-case privacy guarantee of any single mechanism governs the privacy guarantee of the overall process.

Sequential composition theorem

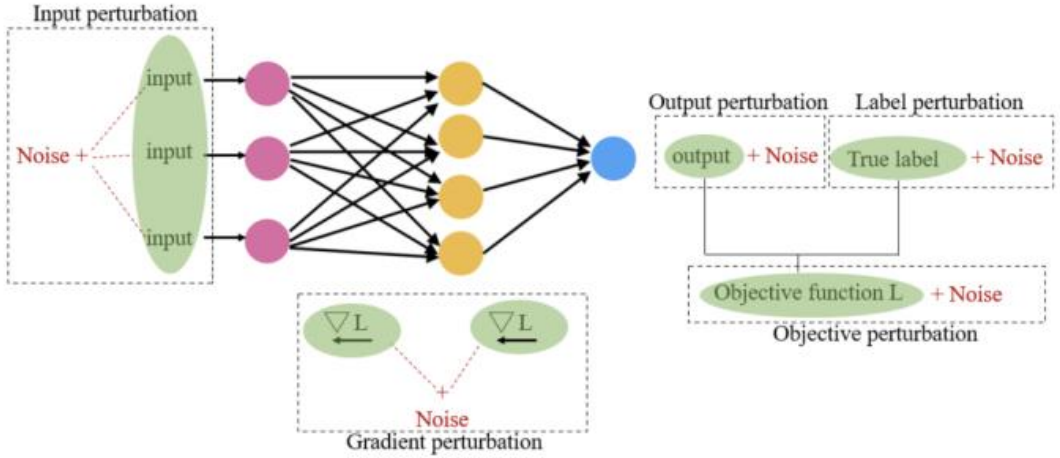
- Suppose you run a sequence of mechanisms:
- $M_1, M_2, \ldots, M_k$
- Each mechanism M_i satisfies:
- $(\varepsilon_i, \delta_i)$-differential privacy
- Then the sequence of all mechanisms together satisfies:
- $(\varepsilon^*, \delta^*)$-differential privacy
- where:
- $\delta^* = \sum_{i=1}^{k} \delta_i$
- ε^* is **at most** $\sum \varepsilon_i$
  - but can be bounded more tightly using advanced composition

Parallel composition theorem

- Suppose:
- You split your dataset into **disjoint subsets** $D_1, D_2, \ldots, D_k$
- You run mechanisms: M_i on D_i
- Each M_i satisfies: $(\varepsilon_i, \delta_i)$-differential privacy
- Then the combined mechanism satisfies:
- $(\varepsilon^*, \delta^*)$-differential privacy where:
- $\varepsilon^* = \max_i \varepsilon_i, \delta^* = \max_i \delta_i$

Where do we implement DP?

- Differential privacy was initially devised for **database analysis**, but it is possible to apply it to deep learning models in several ways.
  - In practice, since we primarily work with deep learning models, we will often refer to datasets rather than databases.
- Figure shows how Differential Privacy can be incorporated in 1) the input data, 2) the gradients, 3) the output data, 4) the labels or even in the loss function



Differential Privacy in Training

Why the training?

- We will introduce a modified version of SGD to get privacy guarantees. This algorithm is known as DP-SGD
- Why?
- Intuitively, through DP-SGD **we bound the amount of information the parameters have about the data**.
- So no matter how many times we use the model parameters to produce outputs, we can’t leak any more information than this bound.
- The alternative would be to train the model normally and then add noise to its outputs.
- However, the privacy loss would accumulate every time you queried the model, so you’d have to add noise proportional to the number of queries… Unfeasible, we will need to add way too much noise

https://mukulrathi.com/privacy-preserving-machine-learning/deep-learning-differential-privacy/

Differentially Private SGD (DP-SGD)

Algorithm 1 Differentially private SGD (Outline)

Input: Examples $\{x_1, \ldots, x_N\}$, loss function $\mathcal{L}(\theta) = \frac{1}{N}\sum_i \mathcal{L}(\theta, x_i)$. Parameters: learning rate η_t, noise scale σ, group size L, gradient norm bound C.

Initialize θ_0 randomly

for $t \in [T]$ **do** // iterate over T mini-batches

Take a random sample L_t with sampling probability L/N

Compute gradient // compute the gradient

For each $i \in L_t$, compute $\mathbf{g}_t(x_i) \leftarrow \nabla_{\theta_t}\mathcal{L}(\theta_t, x_i)$

Clip gradient

$\bar{\mathbf{g}}_t(x_i) \leftarrow \mathbf{g}_t(x_i)/\max\left(1, \frac{\|\mathbf{g}_t(x_i)\|_2}{C}\right)$ // clip the magnitude of the gradients

Add noise

$\tilde{\mathbf{g}}_t \leftarrow \frac{1}{L}\left(\sum_i \bar{\mathbf{g}}_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})\right)$ // add Gaussian random noise to the gradients

Descent

$\theta_{t+1} \leftarrow \theta_t - \eta_t \tilde{\mathbf{g}}_t$

$\varepsilon, \delta \leftarrow$ compute the privacy cost (leakage) so far // compute the privacy cost (leakage) up to t iterations

If $\varepsilon > \varepsilon_{buget}$: then break; // if the cost is over the budget, then stop training

Output θ_T and compute the overall privacy cost (ε, δ) using a privacy accounting method.

// we train a model θ with the privacy budget ε_{budget}

Abadi *et. el*. 2016. Deep Learning with Differential Privacy: https://arxiv.org/pdf/1607.00133.pdf

DP-SGD

- DP-SGD modifies standard SGD in three steps.
  - Dataset size: N, Batch size: B, Sampling rate: $q = \frac{B}{N}$
- **Step 1 — sample minibatch**
  - Sample examples randomly. This creates **privacy amplification by subsampling**.
- **Step 2 — gradient clipping**
  - For each example: $g_i = \nabla L(x_i)$
  - Clip: $\tilde{g}_i = g_i \cdot \min\left(1, \frac{C}{\|g_i\|}\right)$ where C is the clipping norm.
  - This bounds sensitivity.
- **Step 3 — add Gaussian noise**
  - Compute noisy average gradient: $\bar{g} = \frac{1}{B}\left(\sum_i \tilde{g}_i + \mathcal{N}(0, \sigma^2 C^2 I)\right)$
  - Noise multiplier: σ

DP-SGD Gaussian mechanism

DP-SGD uses the **Gaussian mechanism**:

$$M(D) = f(D) + \mathcal{N}(0, \sigma^2 \Delta f^2 I)$$

Noise scale:

- σ= noise multiplier
- Δf = sensitivity

The Gaussian mechanism provides DP guarantees.

Sensitivity in DP-SGD

- DP-SGD controls sensitivity by **gradient clipping**.
- For each example gradient g_i:
- $g_i \leftarrow g_i \cdot \min\left(1, \frac{C}{\|g_i\|_2}\right)$
- Where: C= clipping norm.
- After clipping, the **maximum contribution of one example** is bounded.
- Thus the **L2 sensitivity** of the averaged minibatch gradient becomes:
- $\Delta f = \frac{C}{B}$ where B= minibatch size.
- Some formulations use **sum instead of average**, giving:
- $\Delta f = C$

Noise Added to the Gradient

- Noise sampled is:
- $\mathcal{N}(0, \sigma^2 C^2 I)$ if gradients are **summed**,
- Or $\mathcal{N}\left(0, \sigma^2 \left(\frac{C}{B}\right)^2 I\right)$ if **averaged**.
- The **noise multiplier** σ is the key parameter controlling privacy

DP-SGD key points

- Training involves **many gradient updates**, so we must combine privacy losses.
- **Basic composition**
- If mechanisms $M_1, \ldots, M_T$ each satisfy $(\varepsilon_i, \delta_i)$-DP, then together they satisfy:
- $(\sum_i \varepsilon_i, \sum_i \delta_i)$*−DP*
- PS:
  - This becomes very loose for large T.
  - Deep learning often uses **thousands of steps**, so tighter accounting is needed.

Privacy of One Gaussian Mechanism

- We need to compute the privacy of one step. The Gaussian step provides (ε,δ)-DP if:
- $\sigma = \frac{\sqrt{2\ln(1.25/\delta)}\Delta f}{\varepsilon}$
- Recall:
  - σ= noise std dev
  - Δf= sensitivity
  - ε= privacy loss
  - δ= failure probability
- Rearranging gives the **privacy cost per step**:
- $\varepsilon = \frac{\sqrt{2\ln(1.25/\delta)}\Delta f}{\sigma}$
- However this assumes **one release without subsampling**.
- DP-SGD uses **sampling + composition**, so we need a more refined accounting.

Subsampling Amplification

- In DP-SGD a minibatch is sampled with probability:
- $q = \frac{B}{N}$
- Where
  - B= batch size
  - N= dataset size
- Subsampling reduces privacy leakage because each record participates with probability q. (see Privacy Amplification Theorem below)
- The **effective privacy loss per step** scales approximately as:
- $\varepsilon_{step} \approx q\varepsilon_{Gaussian}$

Privacy Amplification Theorem

- M be (ε, δ)-DP. Each individual is included in the sample with probability q
- Then the composed mechanism satisfies approximately: (ε', δ')−*DP*
- where $\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right)\ and\ \delta' = q\delta$
- Intuition:
- The privacy improves because: an individual might not even be sampled. If they are not sampled → their data has zero influence
- User sampled (prob q)
  - mechanism leaks ε privacy
- User not sampled (prob 1-q)
  - no information leaked
- So the expected privacy loss shrinks.
- For small ε: $\varepsilon' \approx q\varepsilon$

Composition Across Training Steps

- Let:
- $T = \textit{number of iterations}$
- Training for E epochs gives:
- $T = E \cdot \frac{N}{B}$
- From The Algorithmic Foundations of Differential Privacy, **basic composition** states:
- If each mechanism is (ε, δ)-DP, then T compositions give:
- $(T\varepsilon', T\delta)$
- So $\boxed{T\varepsilon' = Tq\varepsilon = E\frac{N}{B}\frac{\sqrt{2\ln(1.25/\delta)}C}{\sigma N}}$

Better bounds

- Strong composition theorem:
  - $(O(q\varepsilon\sqrt{T\log\frac{1}{\delta}}), Tq\delta)$ - differential privacy
- Moment accountant gets us an *even tighter* bound:
  - the overall training process is $(O(q\varepsilon\sqrt{T}), \delta)$ – differential privacy
- Two major improvements:
  - First the budget is now proportional to $\sqrt{T}$ not T, so you can run the algorithm for longer
  - Secondly the failure probability is δ (not $Tq\delta$), so it is *independent* of the number of training steps T.

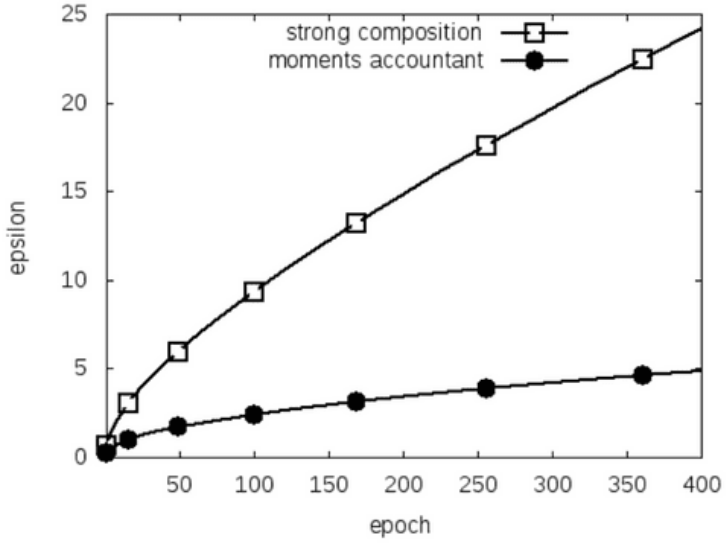


Figure 2: The ε value as a function of epoch E for $q = 0.01$, $\sigma = 4$, $\delta = 10^{-5}$, using the strong composition theorem and the moments accountant respectively.

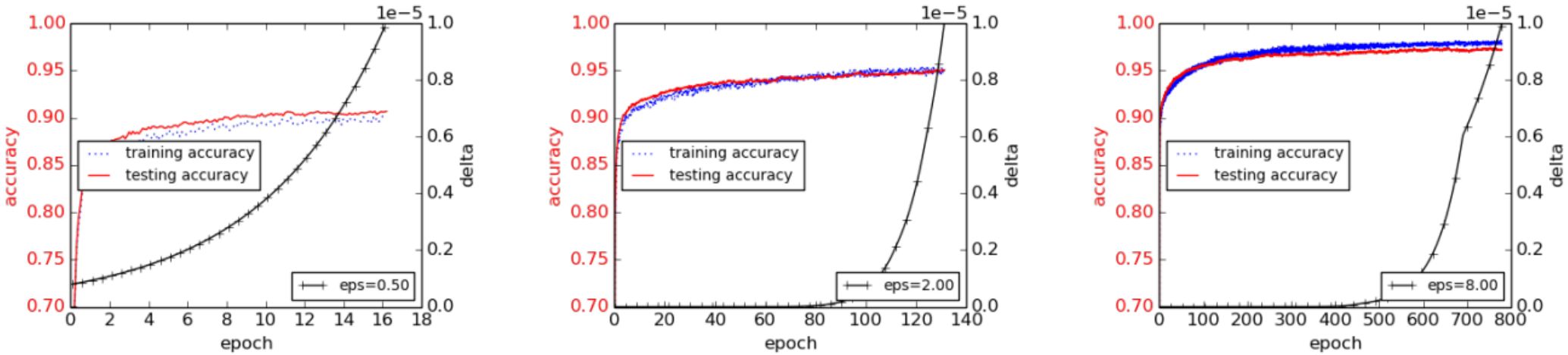
https://mukulrathi.com/privacy-preserving-machine-learning/deep-learning-differential-privacy/

Experiments

- Dataset:
  - MNIST: hand written digits, 28x28 images
- Models: FF with ReLU, cross-entropy loss, optional PCA input layer
  - Baseline: 60 PCA dimension and 1 linear layer of 1000 unit
    - 98.3% on test after 100 epochs
  - DP model: the same architecture with a 60-dimensional PCA projection layer, a single 1,000-unit ReLU hidden layer
    - Clip the gradient norm of each layer at 4.
    - We report results for three choices of the noise scale, which we call small ($\sigma = 2$, $\sigma p = 4$), medium ($\sigma = 4$, $\sigma p = 7$), and large ($\sigma = 8$, $\sigma p = 16$).

Abadi *et. el*. 2016. Deep Learning with Differential Privacy: https://arxiv.org/pdf/1607.00133.pdf

Results



(1) Large noise (2) Medium noise (3) Small noise

Figure 3: Results on the accuracy for different noise levels on the MNIST dataset. In all the experiments, the network uses 60 dimension PCA projection, 1,000 hidden units, and is trained using lot size 600 and clipping threshold 4. The noise levels (σ, σ_p) for training the neural network and for PCA projection are set at (8, 16), (4, 7), and (2, 4), respectively, for the three experiments.

Abadi *et. el*. 2016. Deep Learning with Differential Privacy: https://arxiv.org/pdf/1607.00133.pdf

Remarks

- Despite the significant concerns over privacy, most of today's LLMs abstain from any privacy-specific measure for pre-training, relying instead on publicly available internet data for pre-training.
- While this data is public, it often contains sensitive information, posing risks that might harm individuals and undermine trust. Public data, thus, carries inherent risks yet remains a widely used resource.
- On the other hand, fine-tuning often requires using private datasets that entities prefer to keep confidential to protect sensitive information or maintain a competitive edge.
- We refer to "public training" when the training process involves using datasets that are available to the general public and to "private training" when the training process relies on restricted, confidential data that is not available to the public and must be carefully protected for privacy reasons.
- Differential Privacy can offer a solution for private training minimizing risks associated with data memorization and safeguarding the privacy of individuals in these datasets.
- Working with DP often involves deciding where to inject noise into the training process, typically by adding noise to the gradients. DP-SGD is an optimization strategy, so, in principle, it can be applied to any network

What is the cost?

- We managed to establish bounds for how much is likely that a privacy leaks could occur by **obscuring** the influence of single data points
- This implies that in the learning process, we will capture less information from the data!
- The trade-off will be between privacy and utility

Privacy-Utility tradeoff

Perturbation Locations

Where to inject noise?

Stage	Tecnique
Data input	Noisy training data
Training objective	Modify loss function
Gradient updates	DP-SGD
Model output	Perturb model parameters
Prediction stage	Add noise to predictions

Jarin and Eshete. 2022: DP-UTIL: Comprehensive Utility Analysis of Differential Privacy in Machine Learning https://arxiv.org/pdf/2112.12998.pdf

Privacy-Utility tradeoff

Threat Model

- Attack considered: **Membership Inference Attack from Shokri et al. (shadow models)**
  - Attacker tries to determine: Was a particular data point used in training?

Evaluation Metrics

- **Utility loss**
  - Drop in model accuracy 1-(acc_private / acc_non_private)
- **Privacy leakage**
  - Measured via inference attacks
  - Quantifies the difference between true positive rate and false positive rate of the adversary's inference attack, and its value lies in the range [0,1]
  - We may observe negative values for privacy leakage.
    - The inference attack's false positive rate is greater than true positive rates, which implies that the attack model is likely to detect more non-members as members
- **True revealed samples**
  - Actual number of members whose data is in danger of disclosure when membership inference attack succeeds.

Jarin and Eshete. 2022: DP-UTIL: Comprehensive Utility Analysis of Differential Privacy in Machine Learning https://arxiv.org/pdf/2112.12998.pdf

Privacy-Utility tradeoff

- **Experimental Setup**
- Models tested:
  - Logistic Regression
  - Deep Neural Networks
- Datasets from domains:
  - Vision:
    - CIFAR-10: Images 32x32x3, 10 classes
  - Medical:
    - COVID-19: info on patients and underlying health conditions
    - Binary task for COVID-19 positive or negative
  - Financial:
    - LendingClub-Loan. Lending club is a US peer-to-peer lending company that offers loans.
    - The loan book contain information on the borrower, amount of loan, loan grade, and loan purpose. The dataset contains sensitive features about borrowers.
    - In the accepted loan data, there is a column name grade, which shows the value from A to G
    - The target is to build a classifier that given the other features, classifies accepted loan into 'A' to 'G' grade.

Jarin and Eshete. 2022: DP-UTIL: Comprehensive Utility Analysis of Differential Privacy in Machine Learning https://arxiv.org/pdf/2112.12998.pdf

Privacy-Utility tradeoff

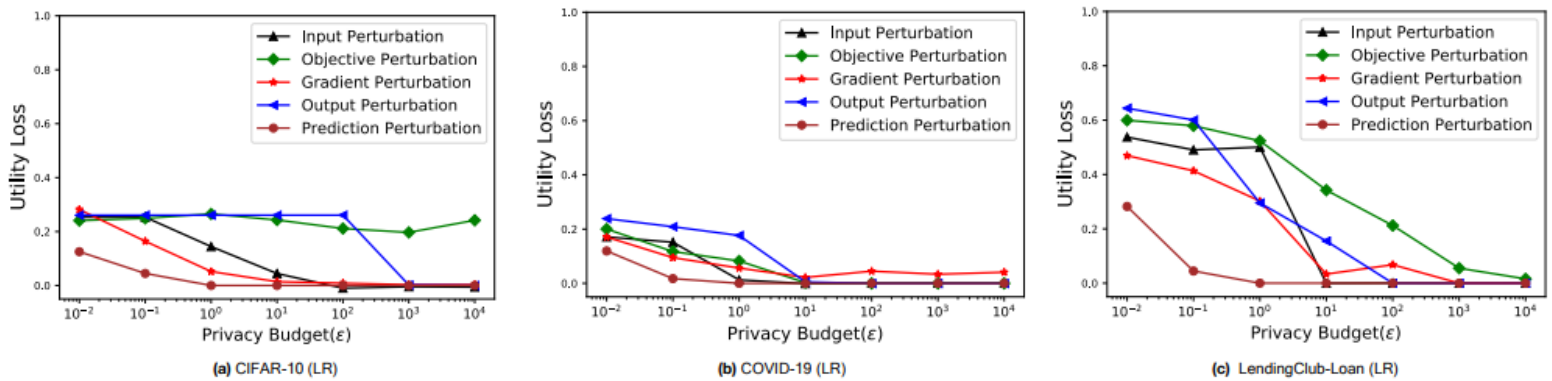


Fig. 2: Utility loss for Logistic Regression (LR).

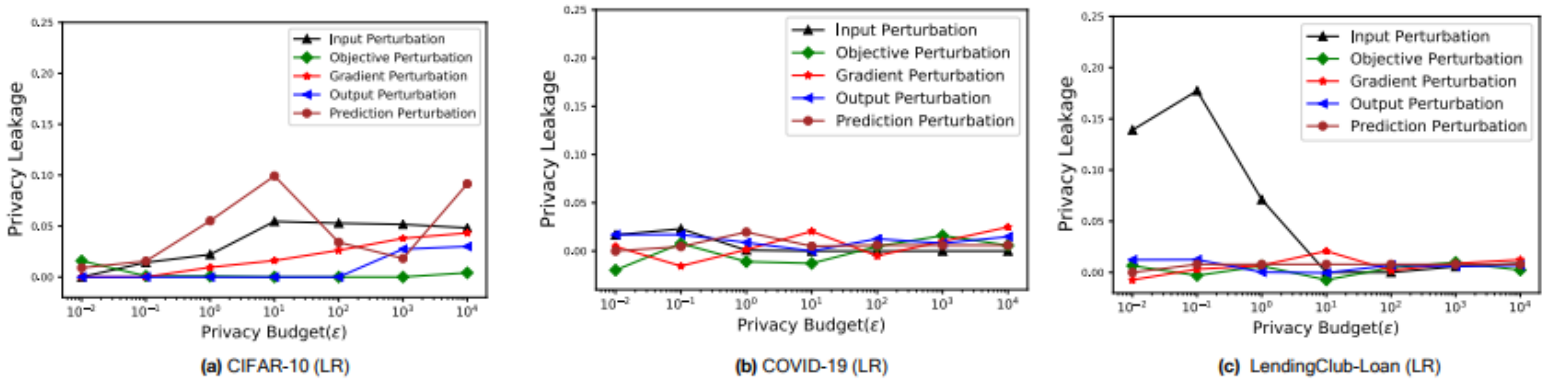


Fig. 4: Privacy leakage for Logistic Regression (LR).

Jarin and Eshete. 2022: DP-UTIL: Comprehensive Utility Analysis of Differential Privacy in Machine Learning https://arxiv.org/pdf/2112.12998.pdf

Privacy-Utility tradeoff

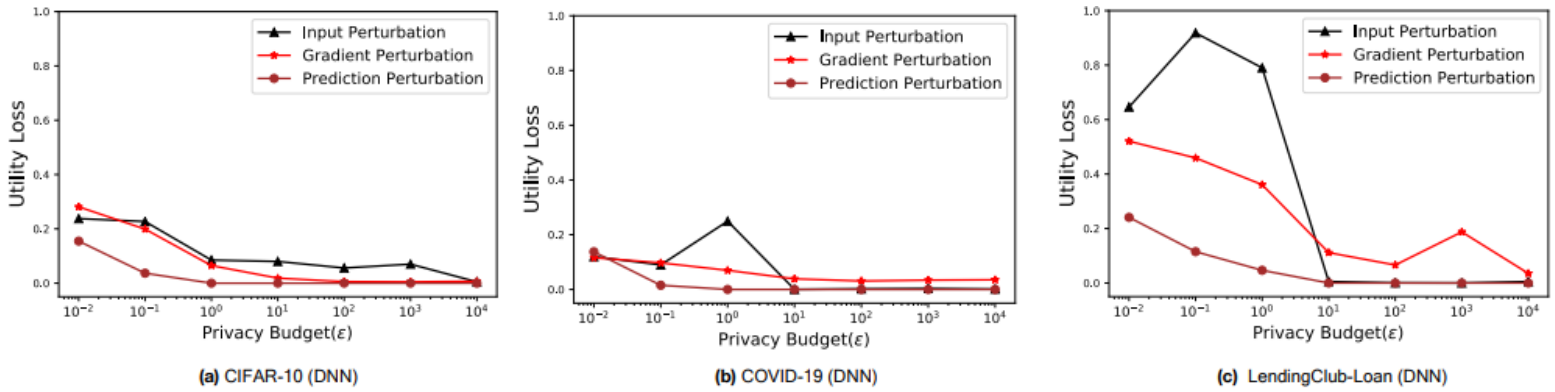


Fig. 3: Utility loss for Deep Neural Network (DNN).

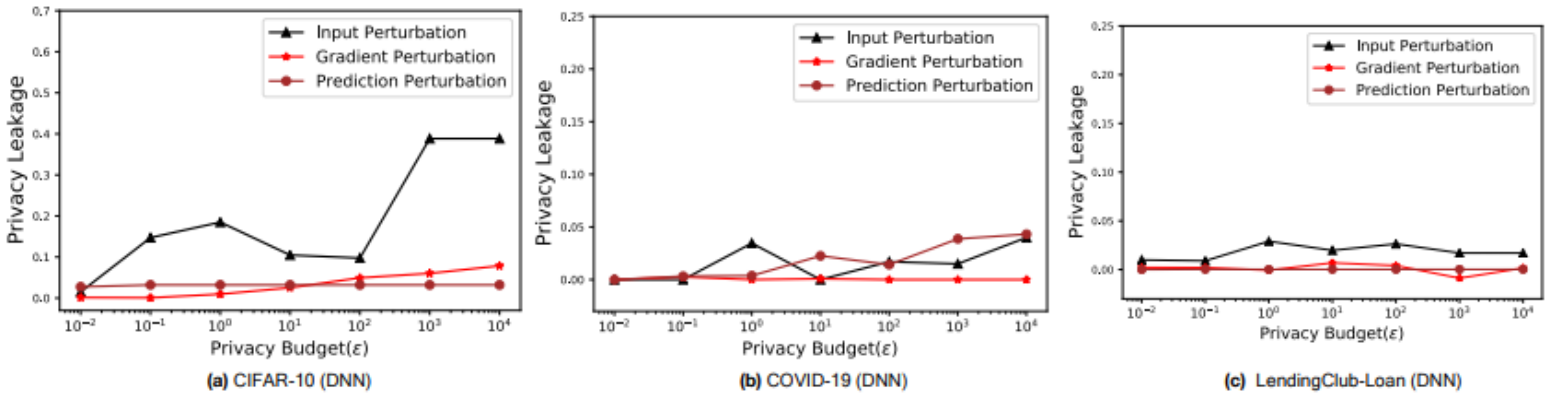
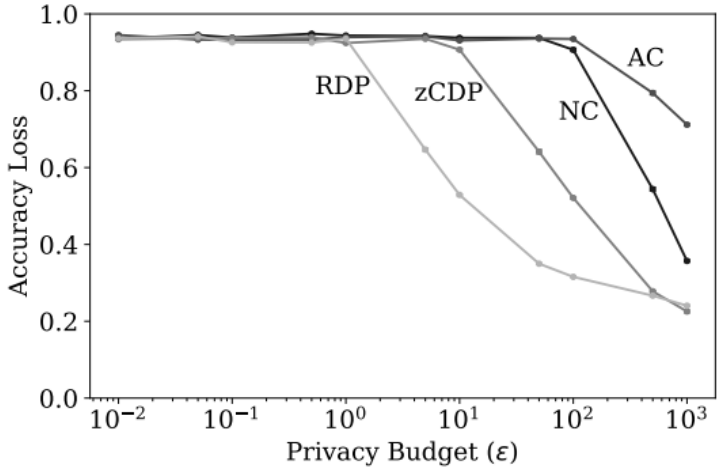


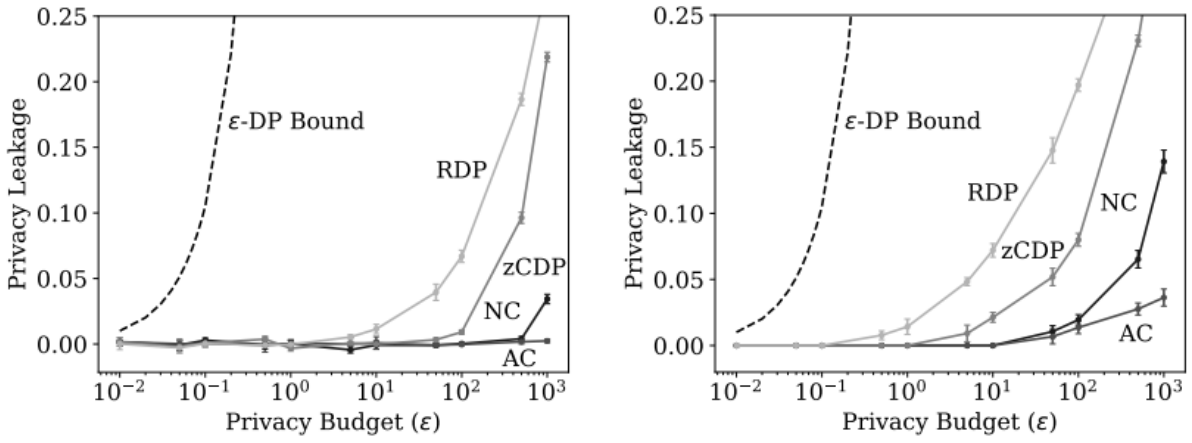
Fig. 5: Privacy leakage for Deep Neural Network (DNN).

Jarin and Eshete. 2022: DP-UTIL: Comprehensive Utility Analysis of Differential Privacy in Machine Learning https://arxiv.org/pdf/2112.12998.pdf

Utility can degrade much faster then expected



(a) CIFAR-100



(a) Shokri et al. membership inference

(b) Yeom et al. membership inference

Figure 6: Inference attacks on neural network (CIFAR-100).

Jayaraman et al., https://www.usenix.org/system/files/sec19-jayaraman.pdf

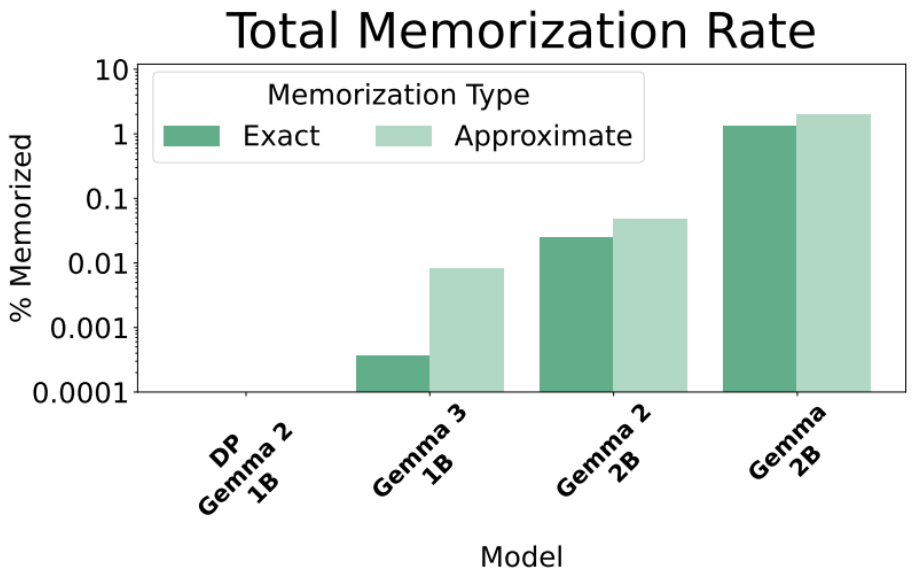
DP at pre-training: VaultGemma

Downstream performance comparison between DP and non-DB models (same architecture, same data)

Training Data Extraction (Extractable Memorization) 50 tokens prefix – 50 token suffixes

Table 2 | A comparison of DP and standard model performance and training configurations.

Model	ARC-C 0 shot	ARC-E 0 shot	HellaSwag 0 shot	PIQA 0 shot	SIQA 0 shot	BoolQ 0 shot	TriviaQA 5 shot
VaultGemma 1B	26.45	51.78	39.09	68.00	46.16	62.04	11.24
Gemma3 1B (PT)	38.31	71.34	61.04	77.37	49.28	68.75	39.75
GPT-2-1.5B	39.78	51.10	47.91	70.51	-	61.80	6.00



VaultGemma team https://arxiv.org/pdf/2510.15001

Fine-tuning with Differential Privacy

- Used DP-SGD to fine-tune LMs for text generation.
- Models: different sizes of GPT2; Data: Open Yelp Dataset.
- Goal: generate synthetic reviews that cannot be traced to the actual reviewer.
- The generated reviews are then used to train classifiers to predict ratings and business categories.
- Metrics:
  - Utility: quality of the synthetic text is assessed by comparing its distribution to the original dataset and the classifier performance against models trained on the original data.
  - Privacy: as in Carlini et al. (2019) injecting canaries inside the training set in order to extract them later on, to prove that the model is capable (or not) of memorizing training data.

Yue et al., https://aclanthology.org/2023.acl-long.74.pdf

Fine-tuning with Differential Privacy

Data Type	Data Generator	Train w/ DP	Rating	Category
Original	-	-	0.7334	0.7752
Synthetic	GPT2	No (ε=∞)	0.6892	0.7584
		Yes (ε=4)	0.6656	0.7478
	GPT2-Medium	No (ε=∞)	0.6878	0.7550
		Yes (ε=4)	0.6756	0.7486
	GPT2-Large	No (ε=∞)	0.7090	0.7576
		Yes (ε=4)	0.6936	0.7568

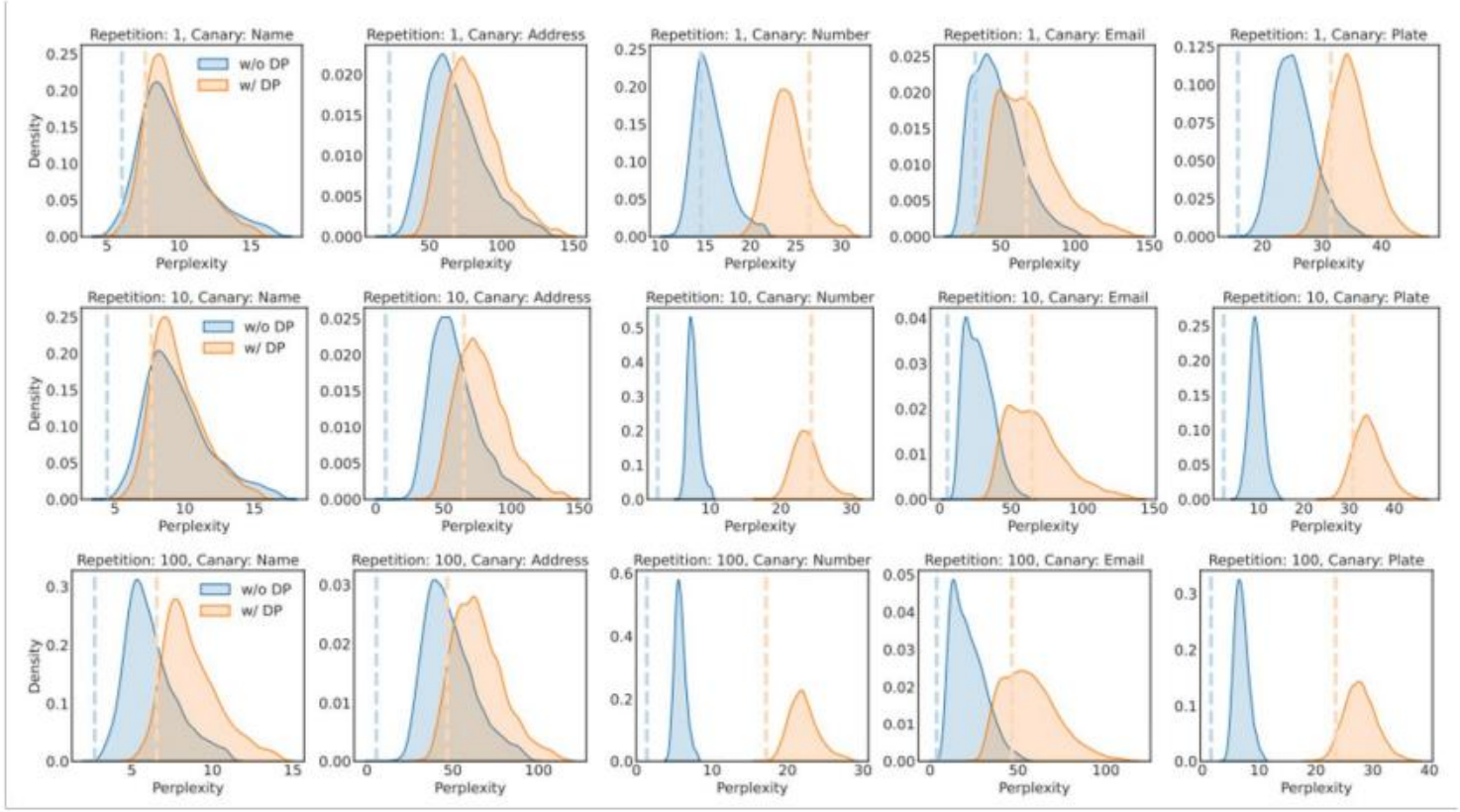
Review Rating Classification

Business Category Classification

The models trained on ***synthetic data generated with DP*** achieve similar performance to the models trained on the ***non-DP counterparts*** and ***original dataset***

Yue et al., https://aclanthology.org/2023.acl-long.74.pdf

Fine-tuning with Differential Privacy



With DP, the private information in canary sequences remains indistinguishable among similar candidates.

Without DP, even when the canary sequence appears only once in the training set, certain canaries (e.g., Address and Plate) still rank highly.

Yue et al., https://aclanthology.org/2023.acl-long.74.pdf

Conclusions and final remarks

- DP offers a solution for designing private model by-design
- It offers theoretical guarantees that the model will not leak private data
  - Please note that under DP definitions and DP-SGD implementation we have that sharing the model **is** different from sharing the data!
- In practice, the utility of the model is at risk:
  - For small epsilons, the risk is that the model is not learning much, hence not memorizing much
- Careful tuning of the hyperparameters and strict evaluation is needed to attest, case by case, the feasibility of this solution
- Positive examples have been seen especially in fine-tuning, and this is valuable as public pre-training, private fine-tuning is a standard pipeline

Machine Unlearning

Acknowledgments

- Sources cited in slides
- All papers discussed
- Some slides from:
  - https://trustworthy-ml-course.github.io/ Lecture 5
  - https://true-lab.ai/courses/MLSec/Sp23/syllabus.html Part IV
  - https://aclanthology.org/2023.acl-long.74.pdf official slides
- Definitions and discussions largely from https://arxiv.org/pdf/2408.05212